VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_Compiler Design

Module 6

Code Generation

Dr. B.V. Baiju,
SCOPE,
Assistant Professor
VIT, Vellore

Register Allocation and Assignment

- Instructions with register operands are faster than those involving memory operands.
- Efficient utilization of registers is vitally important in generating good code.
- One approach to register allocation and assignment is ***to assign specific values in the target program*** to certain registers.
- Example :To assign
  - ***base addresses to one group of registers***
  - ***arithmetic computations to another***
  - ***the top of the stack to a fixed register.***

1. Global Register Allocation

- The code generation algorithm ***used registers to hold values for the duration of a single basic block***.
- However, all live variables were stored at the end of each block.
- ***Keep frequently used value in a fixed register***.
- Global allocation is needed to find out the live ranges of each variable and the area where the variable is used/ defined.
- **Cost of spilling** will depend on the frequencies and location of uses.
- Register allocation depends on
  - ***Size of live range***
  - ***Number of uses / definitions***
  - ***Frequency of execution***
  - ***Number of loads / stores needed***
  - ***Cost of load/ stores needed***

2. Usage Counts

- Usage count is the ***count for the use of some variable x in some register*** used in the basic block.
- Usage count gives the idea about ***how many units of cost can be saved*** by selecting a specific variable for global register allocation.
- The approximate formula for usage count for the loop L in some basic block B can be given as

$$\sum_{\text{blocks } B \text{ in } L} (use(x,B) + 2 * live(x,B))$$

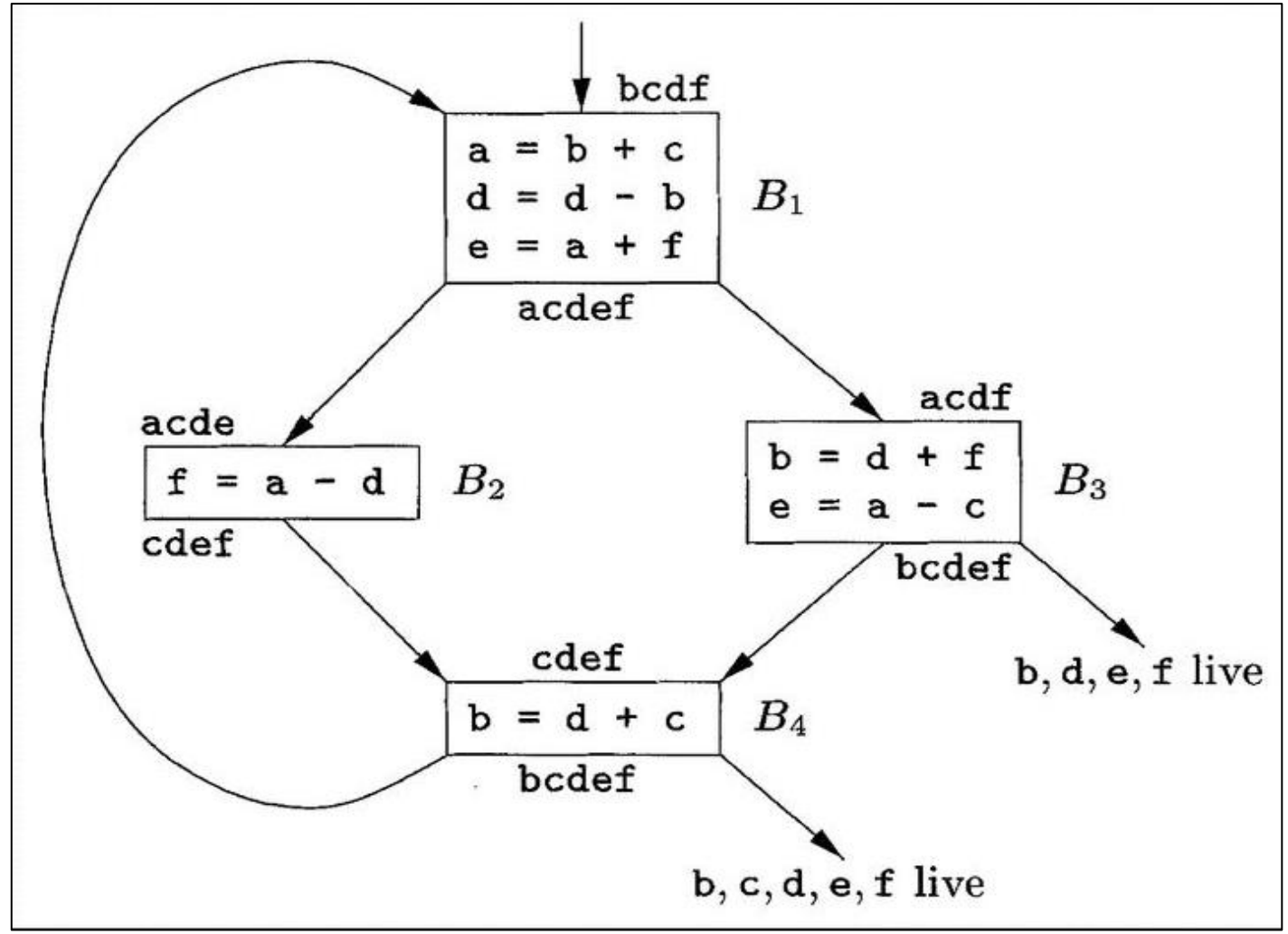
- **use(x, B)** is the number of times x is used in B and is not preceded by an assignment to x in the same block.
- **live(x,B)** is **1** if x is live on exit from B and is assigned a value in B.
- **live(x,B)** is **0** otherwise.

- If x is in register, we can save 1 unit of cost for each reference of x, that is not preceded by an assignment to x in the same block
- We save 2 units if we can avoid a store of x at the end of a block and in which x is assigned a value.

- Consider the basic blocks in the inner loop where jump and conditional jump statements have been omitted.
- Assume registers **RO, R1,** and **R2** are allocated to hold values throughout the loop.

$$\sum_{\text{blocks } B \text{ in } L} (use(x, B) + 2 * live(x, B))$$

	B1	B2	B3	B4	Total Cost
a	0+2	1+0	1+0	0+0	4
b	2+0	0+0	0+2	0+2	6
c	1+0	0+0	1+0	1+0	3
d	1+2	1+0	1+0	1+0	6
e	0+2	0+0	0+2	0+0	4
f	1+0	0+2	1+0	0+0	4



- **use(x, B)** is the number of times x is used in B and is not preceded by an assignment to x in the same block.
- **live(x,B)** is **1** if x is live on exit from B and is assigned a value in B.
- **live(x,B)** is **0** otherwise.

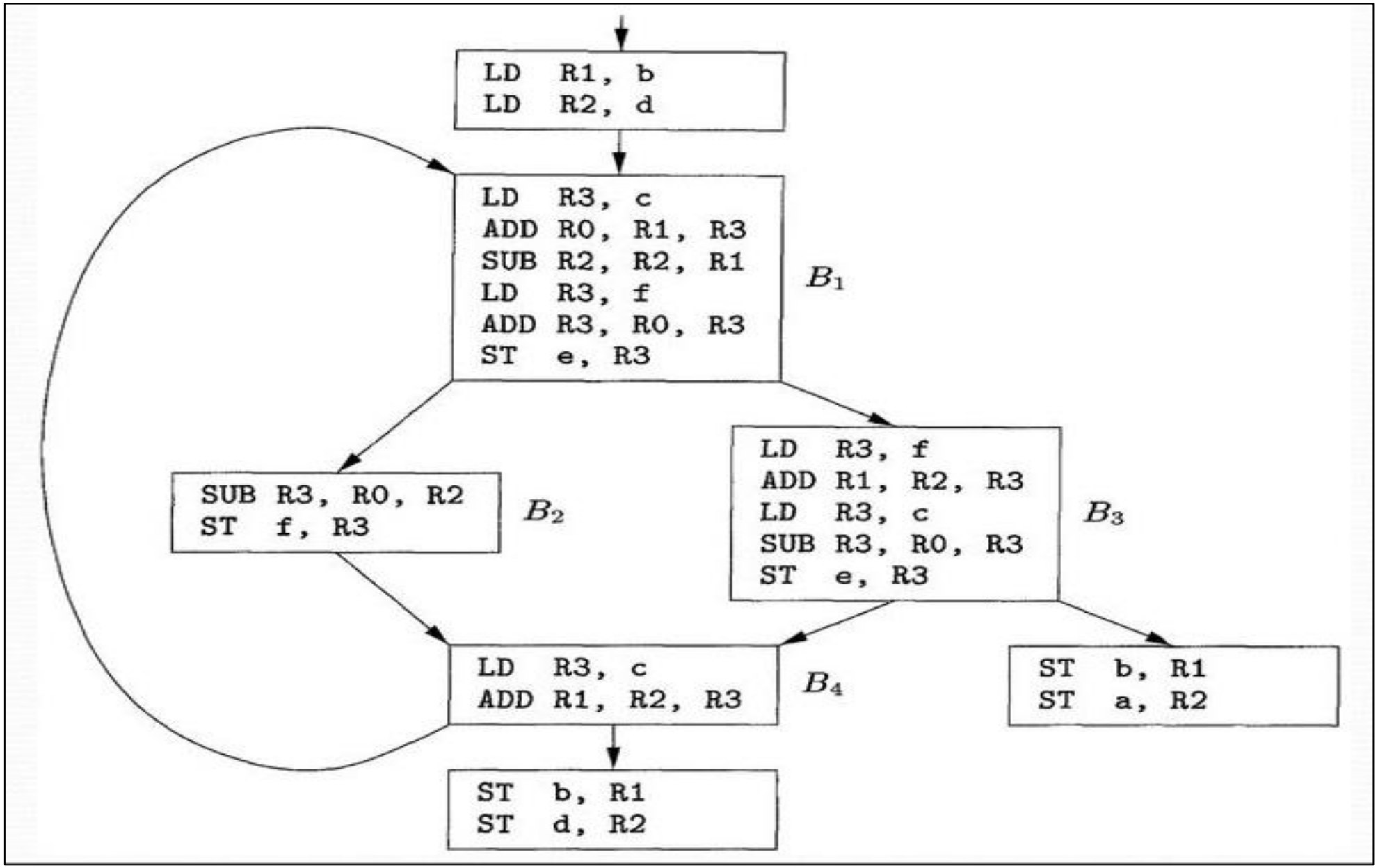
- If a is stored in any one register we can save 4 units of cost.
- If b is stored in any one register we can save 6 units of cost.
- If c is stored in any one register we can save 3 units of cost.
- If d is stored in any one register we can save 6 units of cost.
- If e is stored in any one register we can save 4 units of cost.
- If f is stored in any one register we can save 4 units of cost.

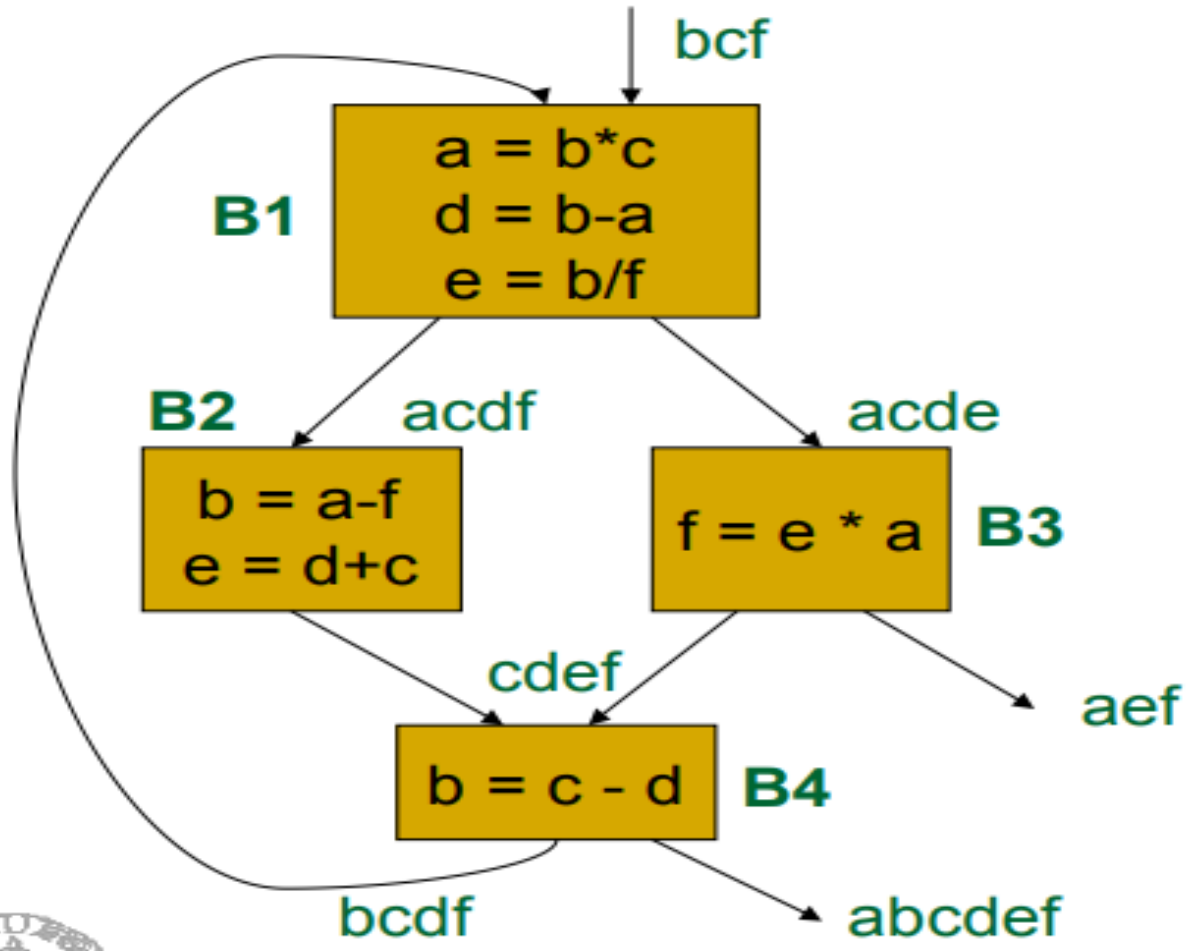
- We can use R0,R1, and R2 as fixed register and we can assign

R0	R1	R2
b	d	a/e/f

variables	Total Cost
a	4
b	6
c	3
d	6
e	4
f	4

Code sequence using global register assignment





Savings for the variables

	B1	B2	B3	B4	
a:	(0+2)+	(1+0)+	(1+0)+	(0+0)	= 4
b:	(3+0)+	(0+0)+	(0+0)+	(0+2)	= 5
c:	(1+0)+	(1+0)+	(0+0)+	(1+0)	= 3
d:	(0+2)+	(1+0)+	(0+0)+	(1+0)	= 4
e:	(0+2)+	(0+2)+	(1+0)+	(0+0)	= 5
f:	(1+0)+	(1+0)+	(0+2)+	(0+0)	= 4

If there are 3 registers, they will be allocated to the variables, a, b, and e

3. Register Assignment for Outer Loops

- If an outer loop L_1 contains an inner loop L_2, the names allocated registers in L_2 need not be allocated registers in L_1 — L_2.
- If x is allocated to register in L_2 then load x on entrance of L_2 and store x on exit from L_2

```
L1: for i = 1 to N do
  // Operations in L1
    L2: for j = 1 to M do
          x = x + j
  // Operations in L2 that use x
        end for
  // Operations in L1
   end for
```

```
L1: for i = 1 to N do
      load x into R1   // Load x into register R1
    L2: for j = 1 to M do
            R1 = R1 + j      // Use R1 for x in L2
        end for
      store R1 into x  // Store R1 back to x after L2 ends
      // Operations in L1 continue
   end for
```

Allocate Register for x in *L2*

- Allocate a register, say *R*1, to variable *x*x in the inner loop *L*2.

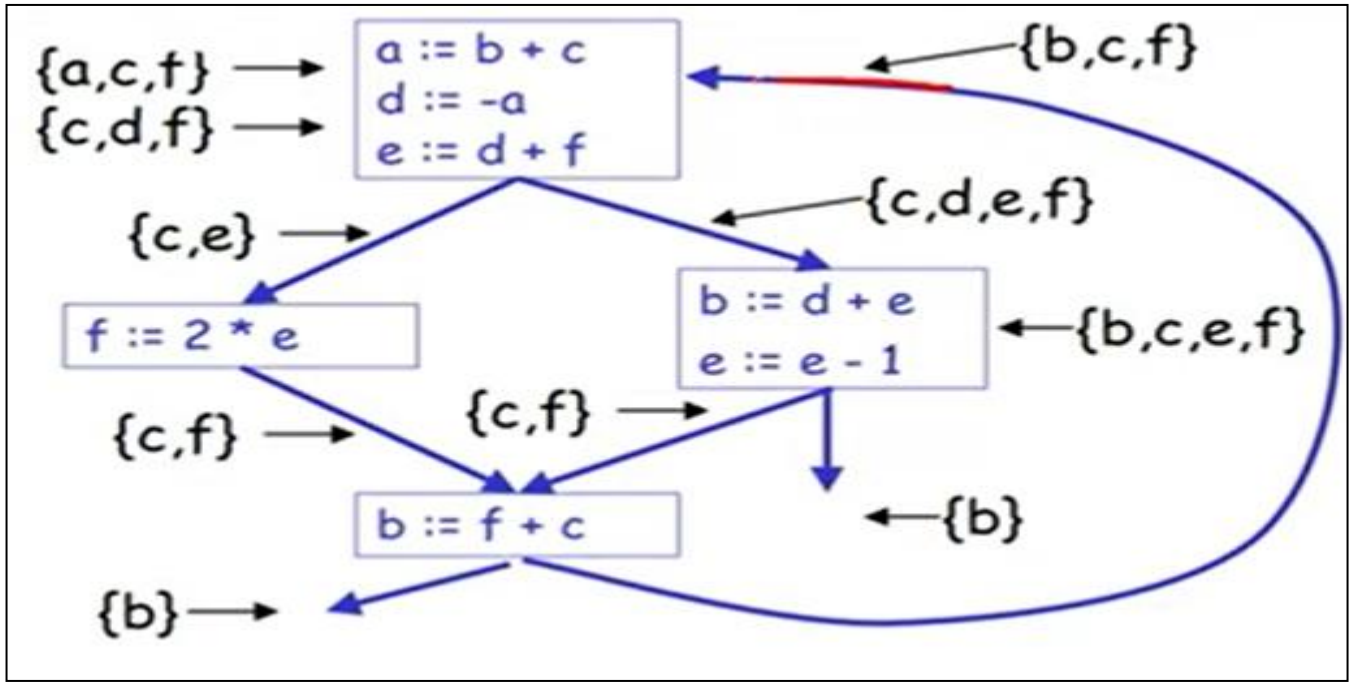
Load and Store for *x*:

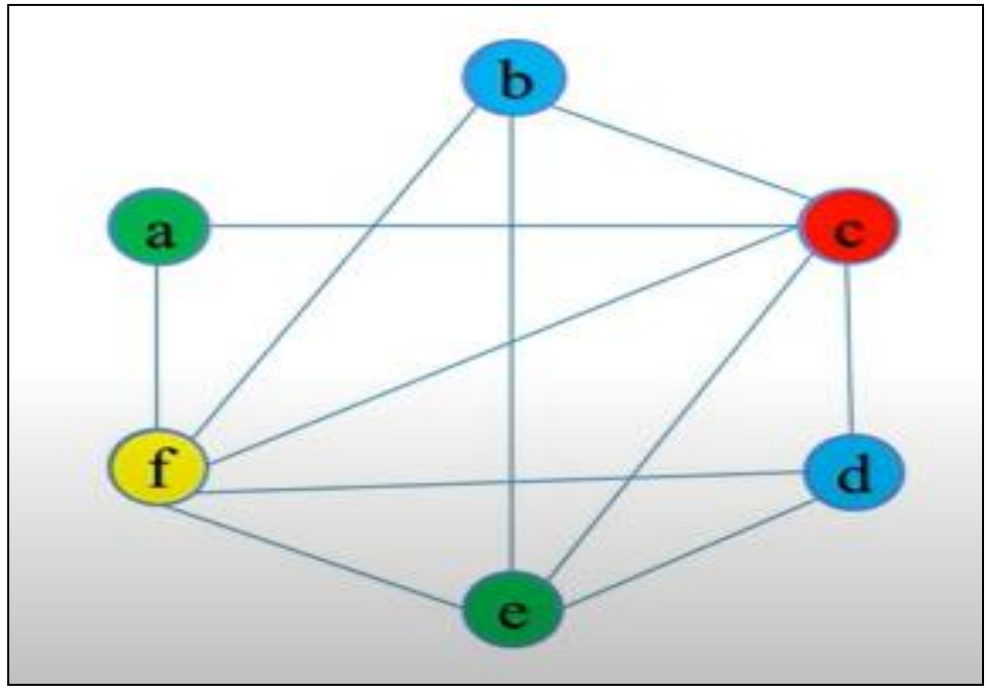
- Load *x* into *R*1 at the entrance of *L*2 (when *L*2 begins).
- Perform all operations on *x* within *L*2 using *R*1.
- Store *x* from *R*1back to memory at the exit of *L*2 (when *L*2 ends).

4. Register Allocation by Graph Coloring

- Graph coloring is a simple, systematic technique for ***allocating registers*** and ***managing register spills***
- Two passes are used in this method
- **First Pass** : Target-machine instructions are selected as though there are an infinite number of symbolic registers; in effect
  - ***names used in the intermediate code become names of registers***
  - ***three-address instructions become machine-language instructions.***
- Once the instructions have been selected, a second pass assigns physical registers to symbolic ones.
- The goal is to find an assignment that minimizes the cost of spills.
- **Second Pass:** For each procedure a ***register-interference graph*** is constructed in which the nodes are symbolic registers and an edge connects two nodes if one is live at a point where the other is defined.
- Attempt to color the graph using as few colors as possible (each color represents a unique physical register). Nodes connected by an edge must have different colors, ensuring that interfering registers receive different physical registers.
- If the graph cannot be colored with the available number of physical registers, select nodes to "spill" (move to memory) to free up registers and retry the coloring.

- Need to assign colors(registers) to graph nodes (temporaries)





- Symbolic Registers or temporaries = { a,b,c,d,e,f}
- Nodes according to descending order of degree
- c, f, e, b, d, a = 5, 5, 4, 3,3 ,2
- 4 colors required = 4 registers required